

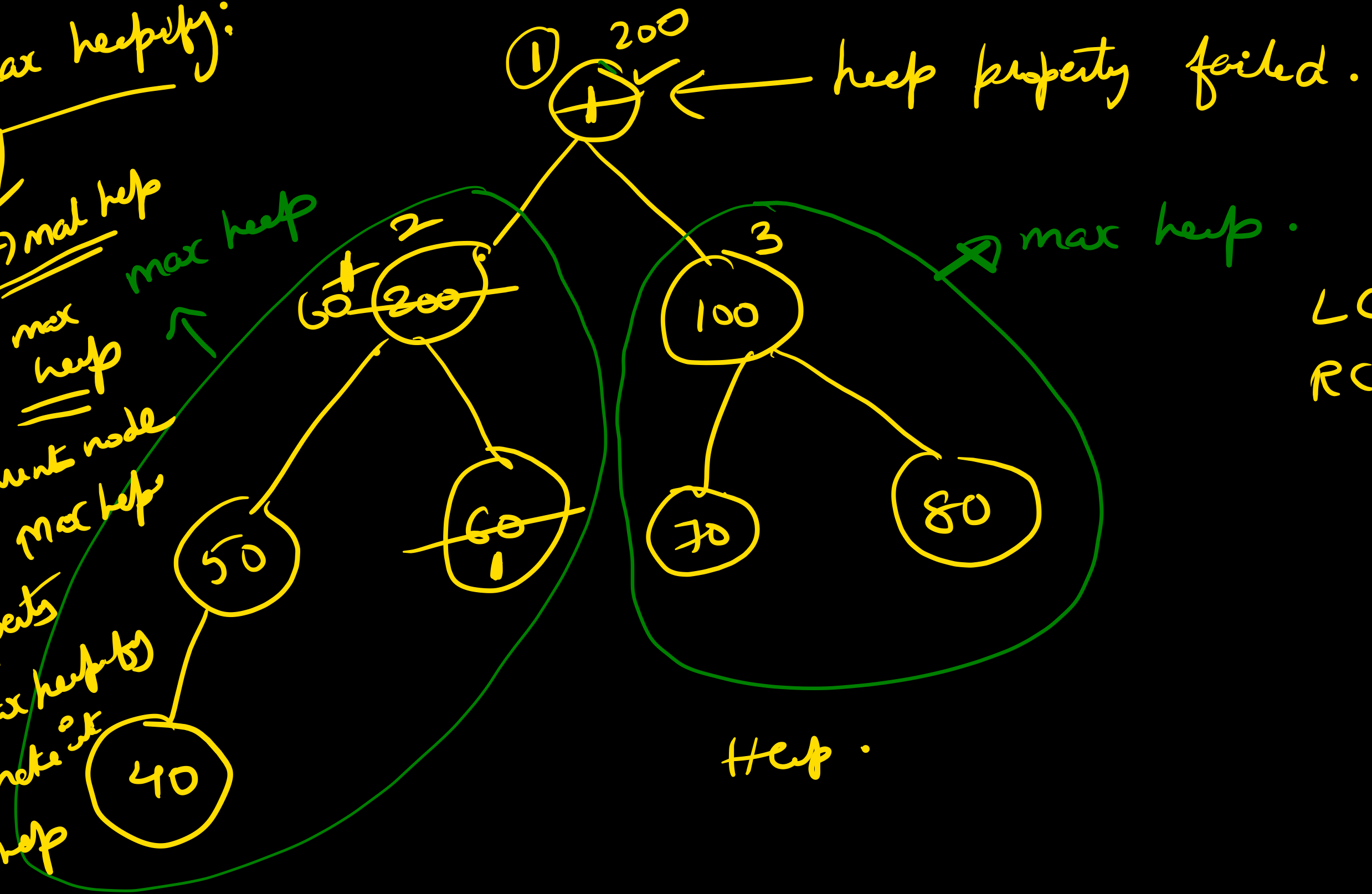
max heap property:

LST $\rightarrow$ max heap

RST $\rightarrow$ max heap

But parent node failed max heap property

Then max heap will make it max heap



$$LC(i) = 2 \times i$$
$$RC(i) = 2 \times i + 1$$

max heapify (A, i)

{

$l = 2i;$

$r = 2i+1;$

if ($l \leq A.heapsize$ and $A[l] > A[i]$)
 $layer = l;$

else ($layer = r$);

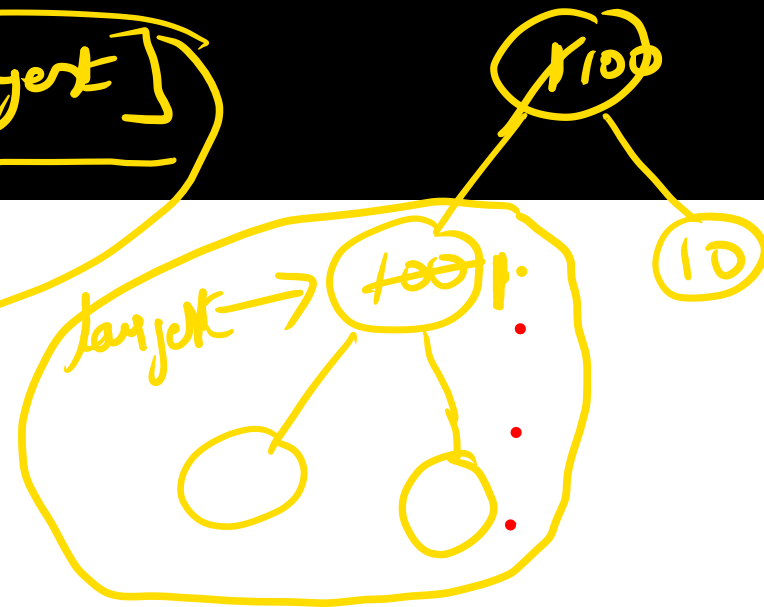
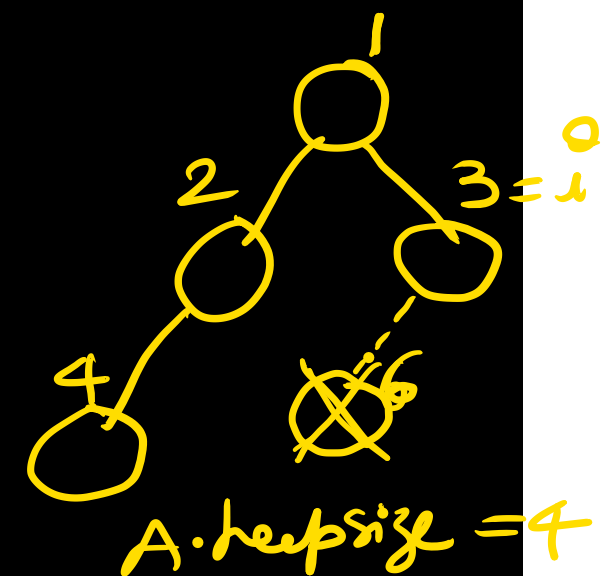
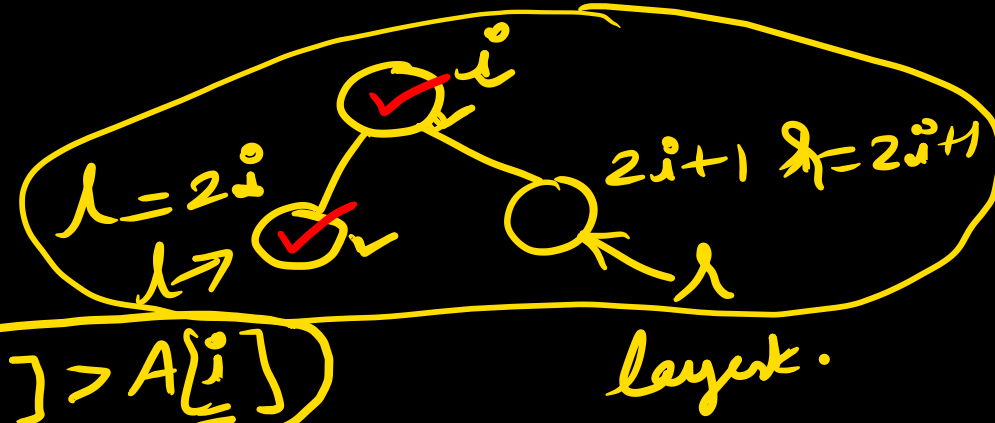
if ($r \leq A.heapsize$ and $A[r] > A[layer]$)
 $layer = r;$

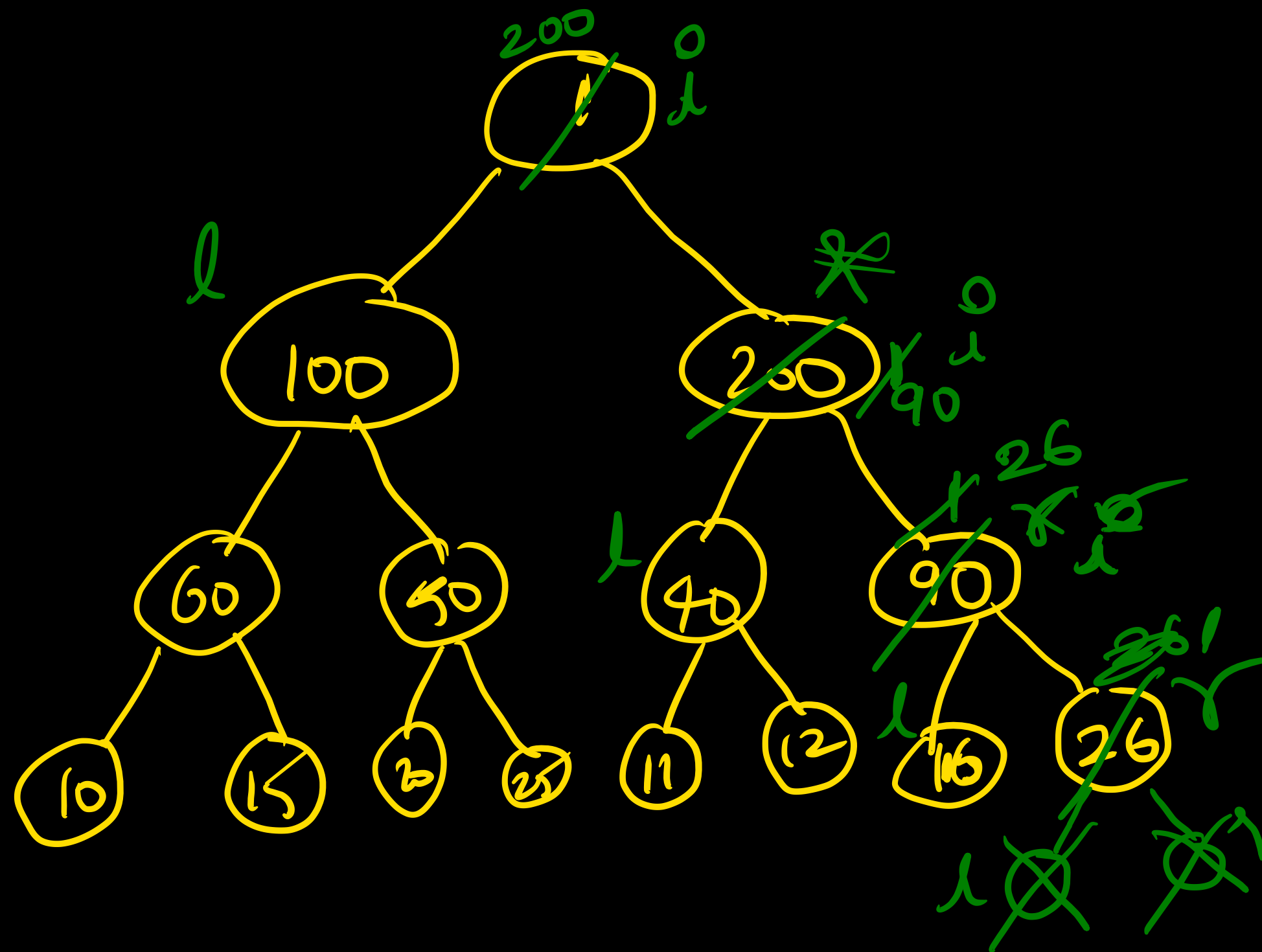
if ($layer \neq i$)

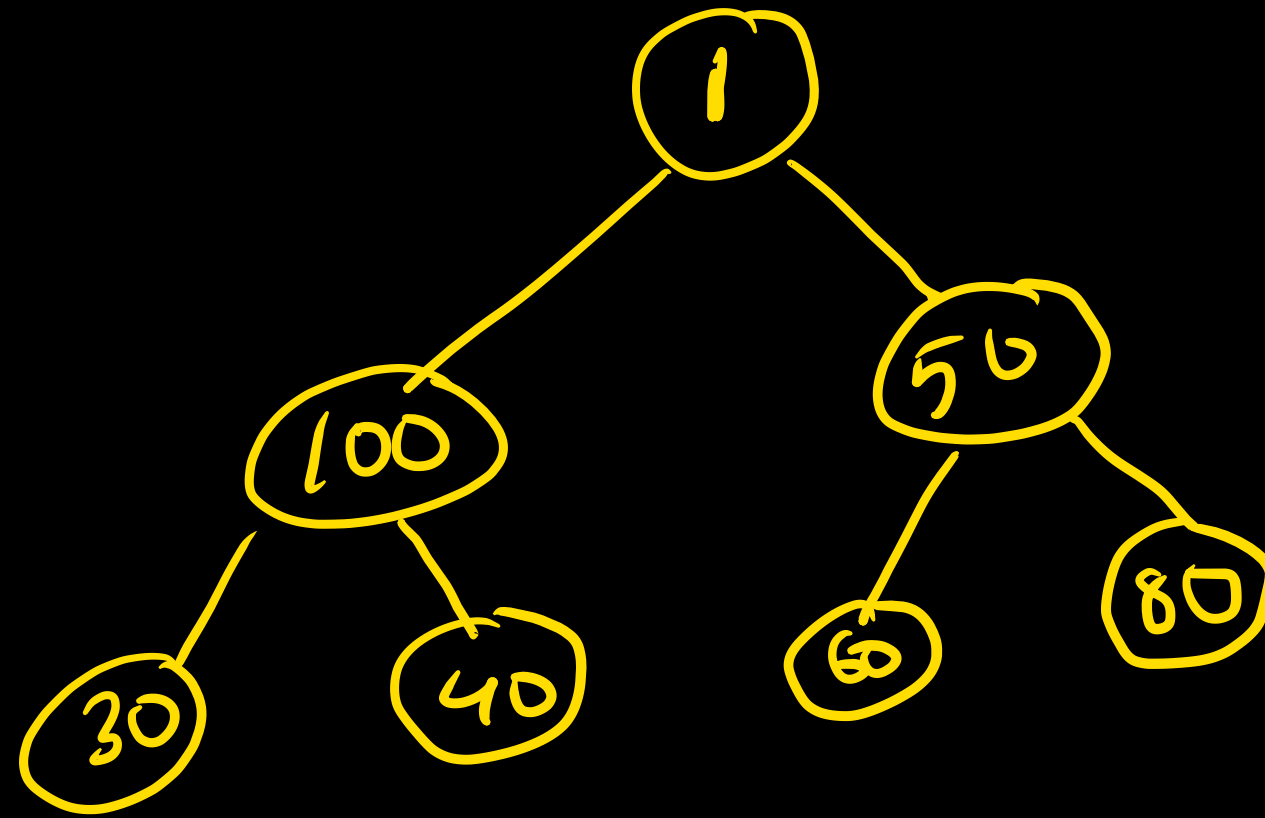
exchange $A[i]$ with $A[layer]$

max heapify (A, layer)

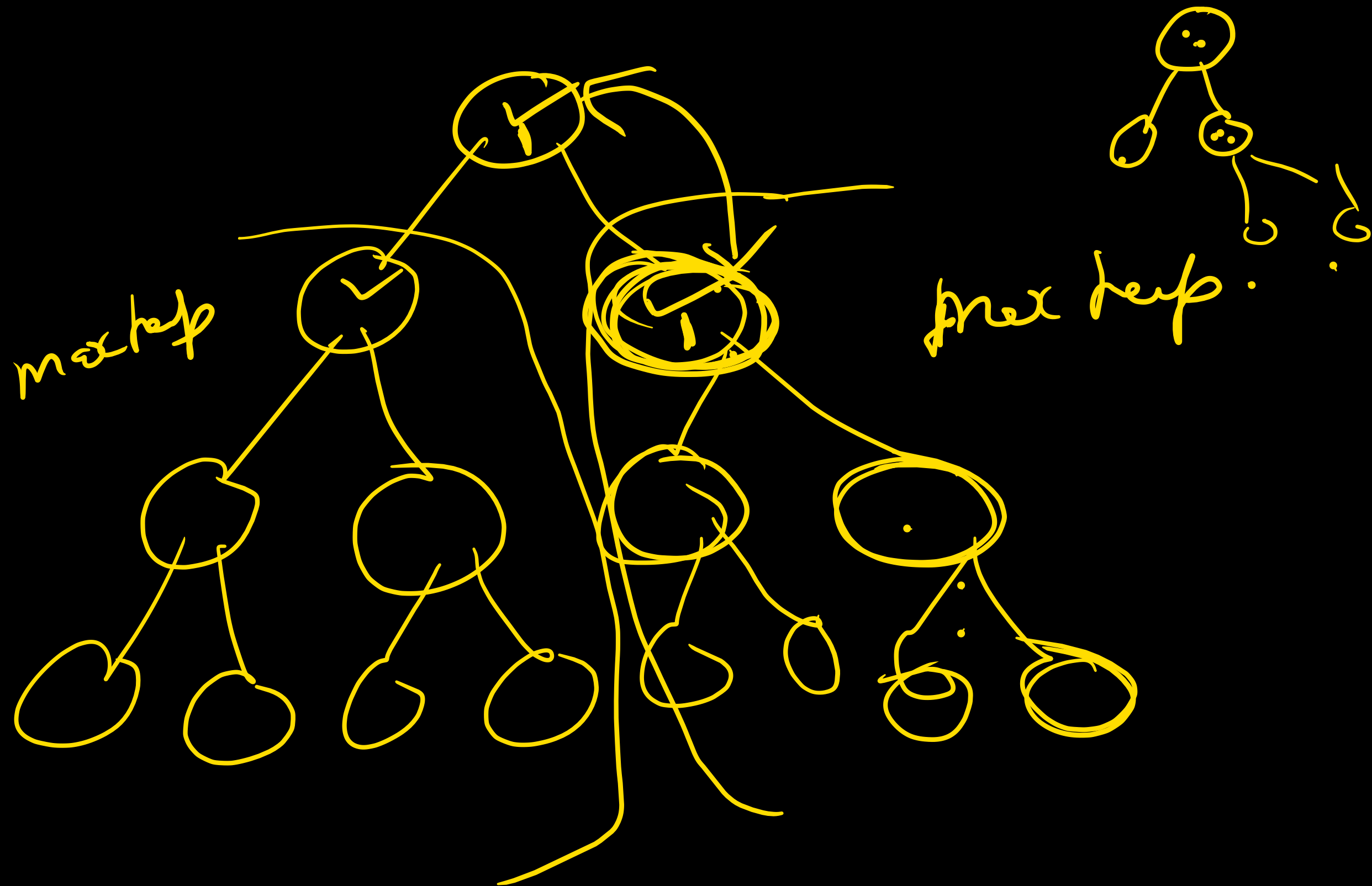
}

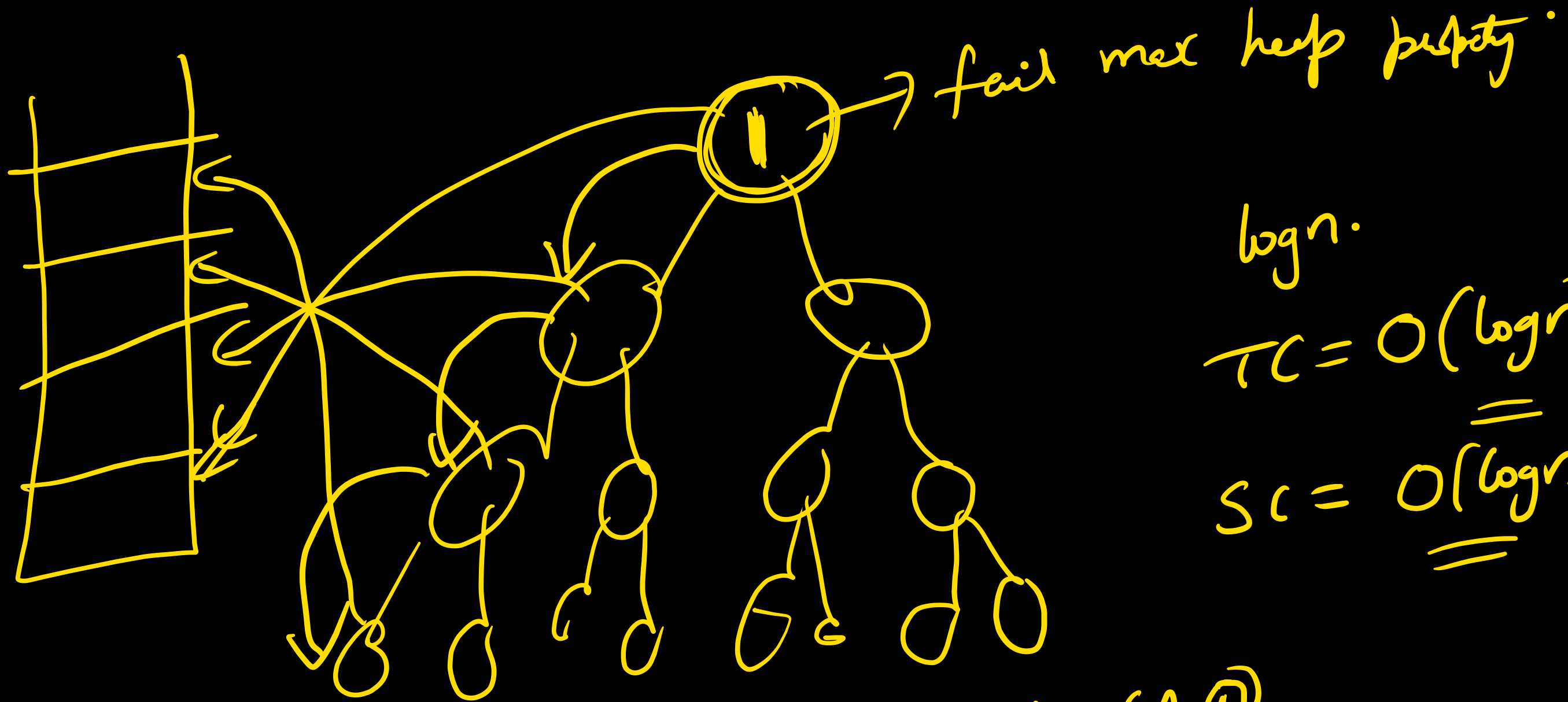






X max heap.





$$\log n.$$

$$TC = O(\log n)$$

$$SC = \underline{\underline{O(\log n)}}$$

max heapify(A, 1)

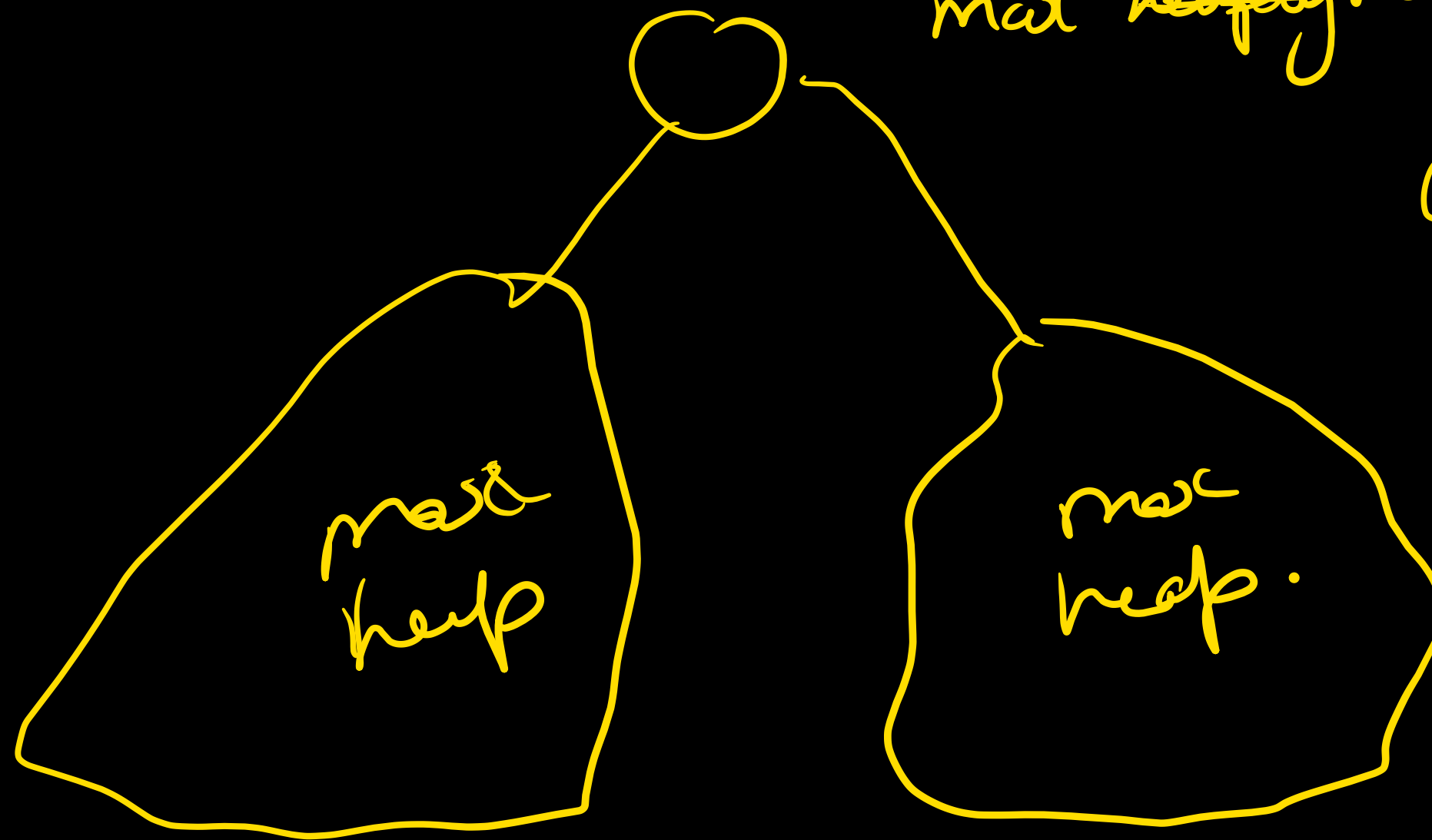
↓

A(2)

A(4)

A(8)

max ~~heapify~~ heapify.



(Q) How to build
max heap
from ~~set~~
Sketch.

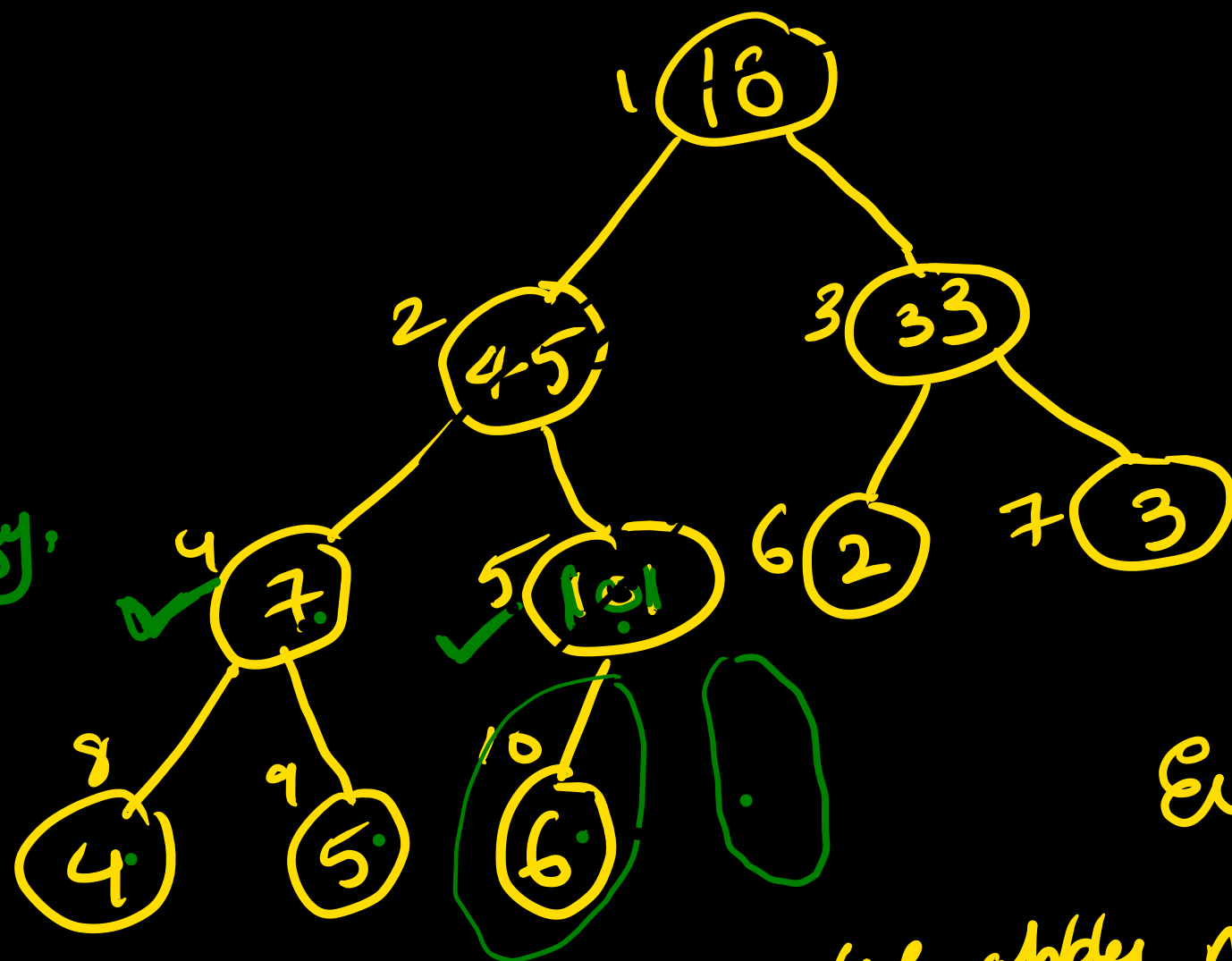
array

18	45	33	7	101	2	3	4	5	6
1	2	3	4	5	6	7	8	9	10

Almost
complete
binary
tree

Apply max heapify.

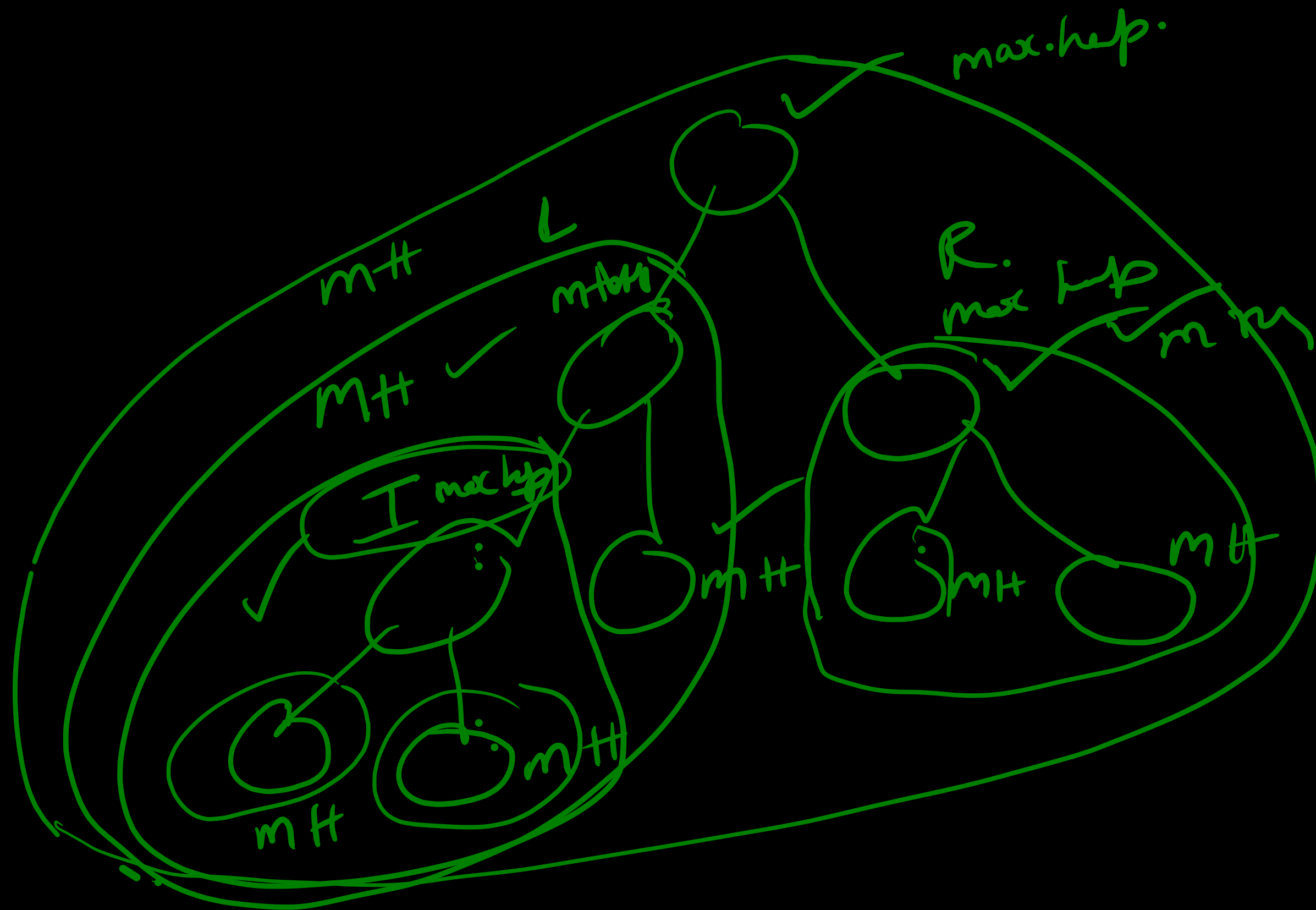
5, 4, 3, 2, 1

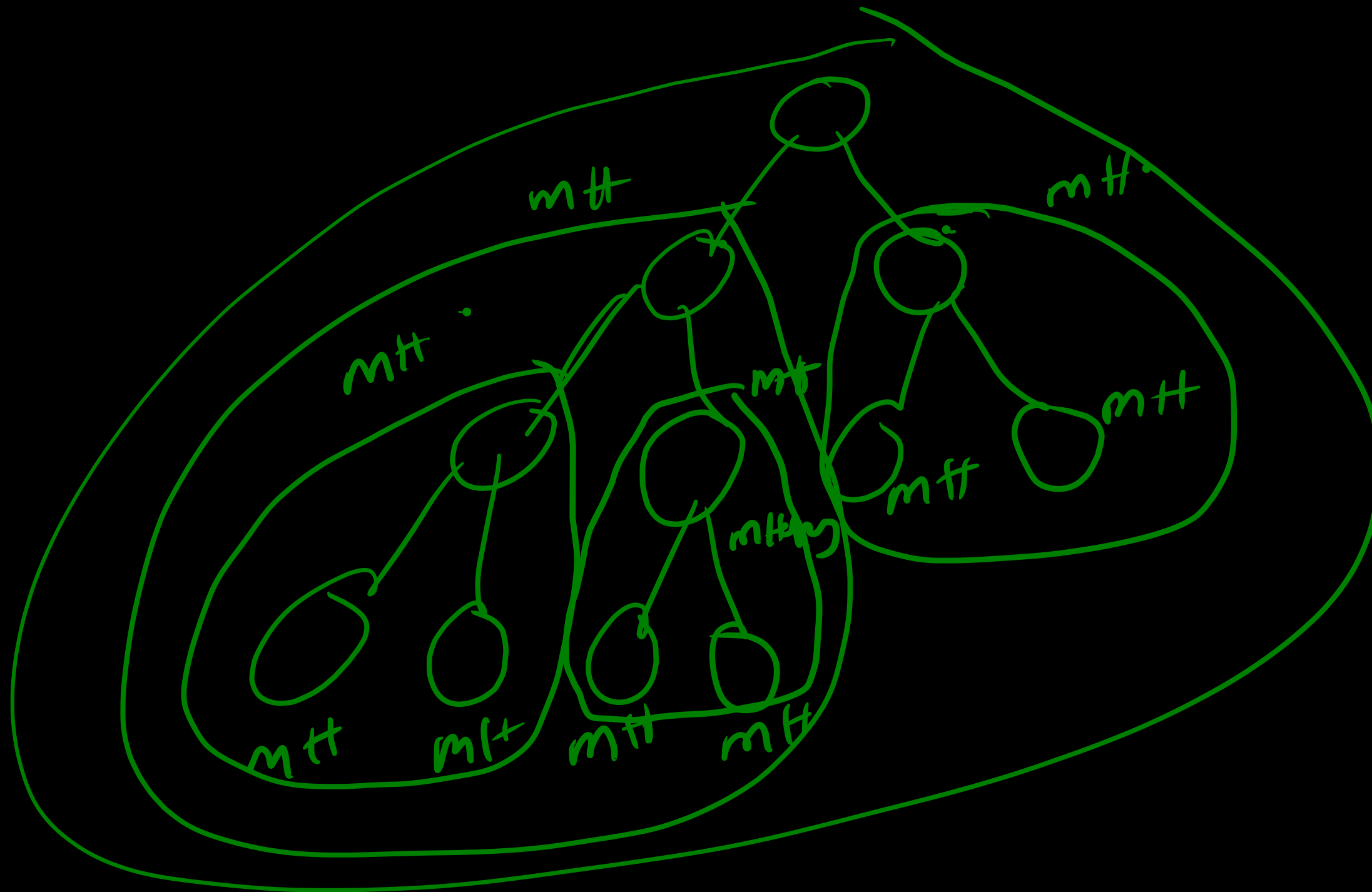


$$\begin{aligned} \text{leaves} &= \left\lfloor \frac{\text{length}}{2} \right\rfloor + 1 \text{ to } n \\ &= \left\lfloor \frac{10}{2} \right\rfloor + 1 \text{ to } 10 \\ &= \underline{\underline{6 \text{ to } 10}} \end{aligned}$$

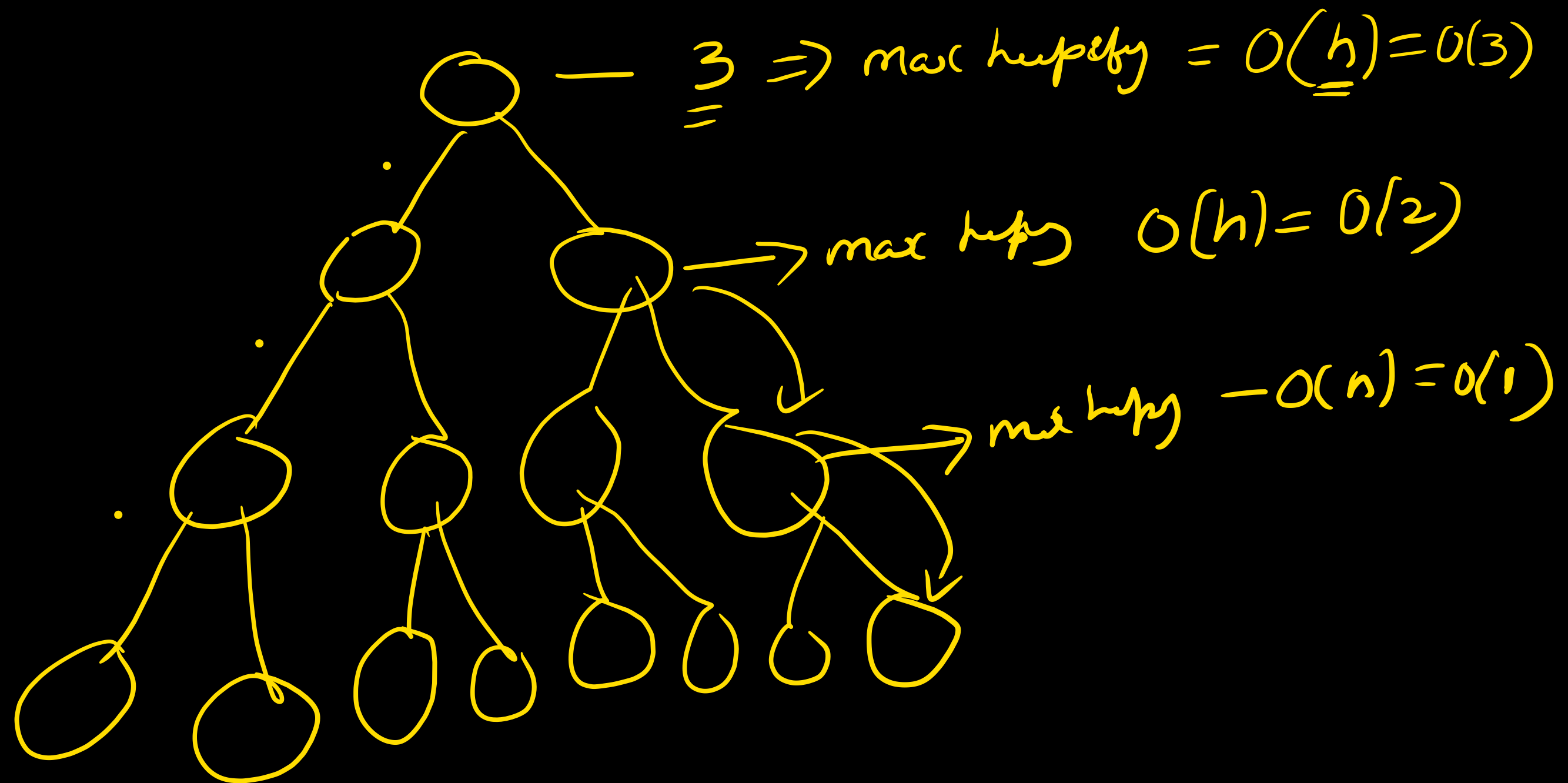
Every leaf is a max heap.

We apply max heapify
on non nonleaves from end

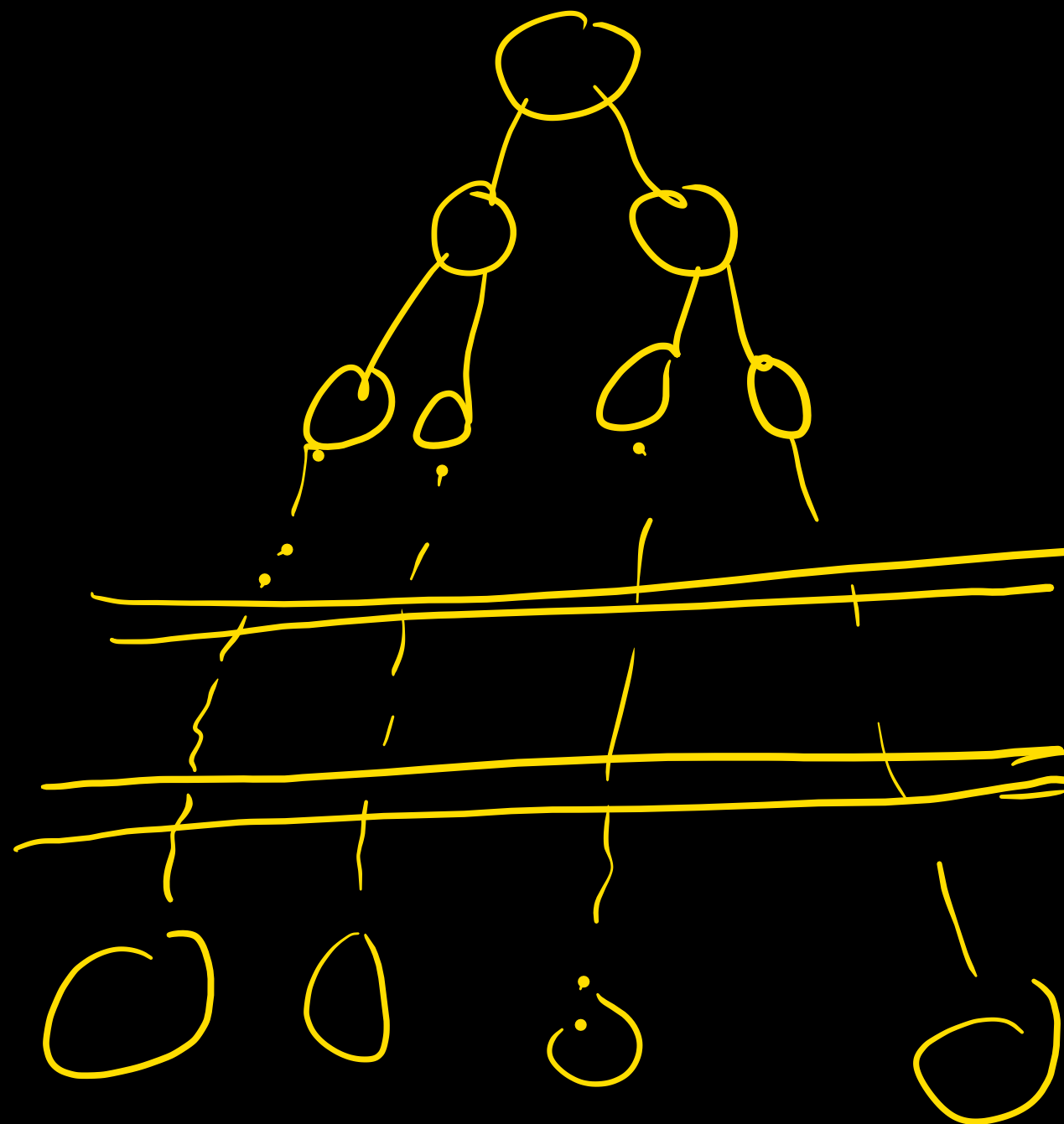




~~You are
wrong..~~



But
max heapify
 $= O(h) =$

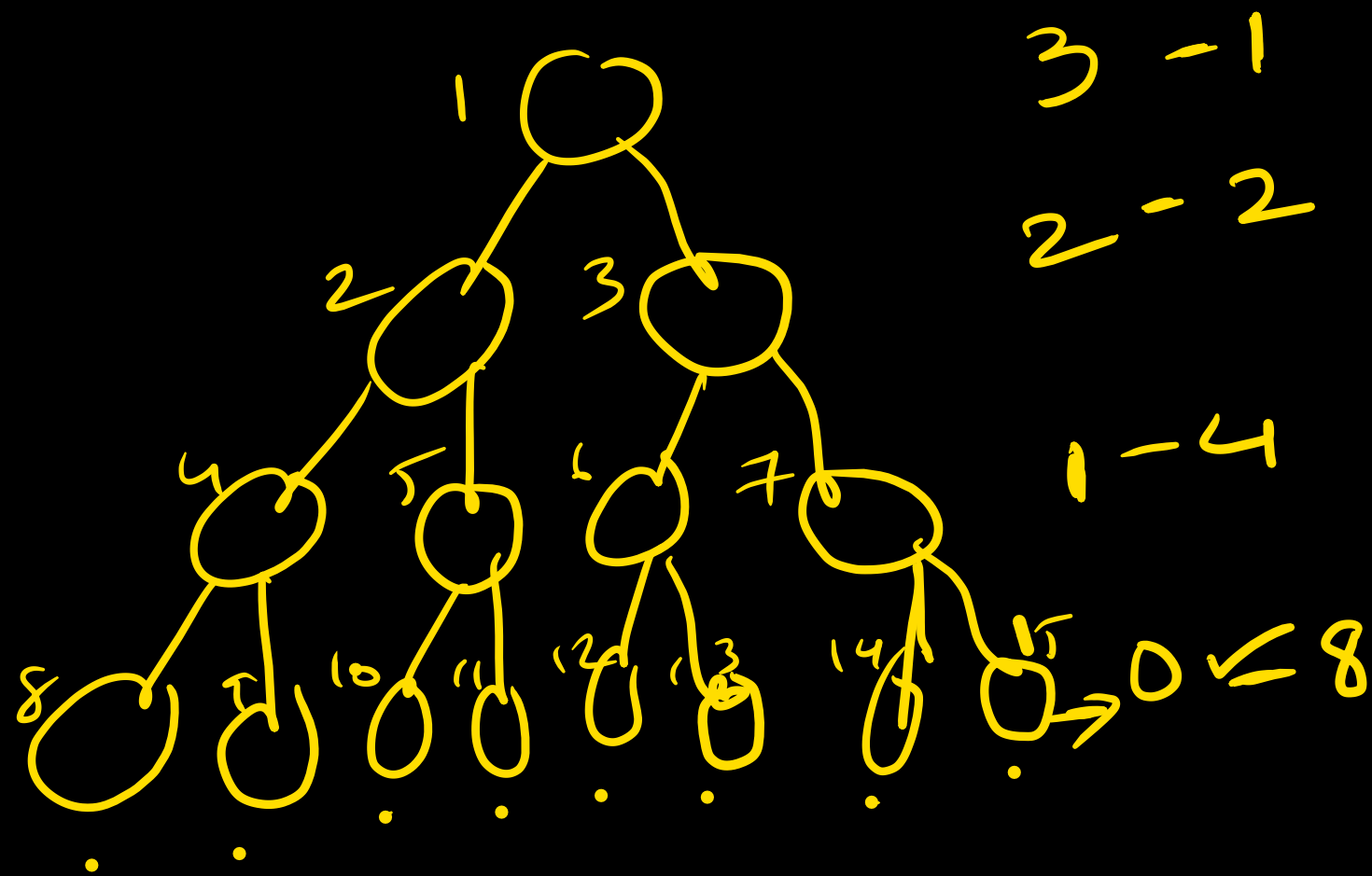


height = h

$K * O(h)$ ✓

$1 * O(h-1)$ ✓

$n * \log n$
↓
max heapify
is always $\log n$



$$3 - 1$$

$$2 - 2$$

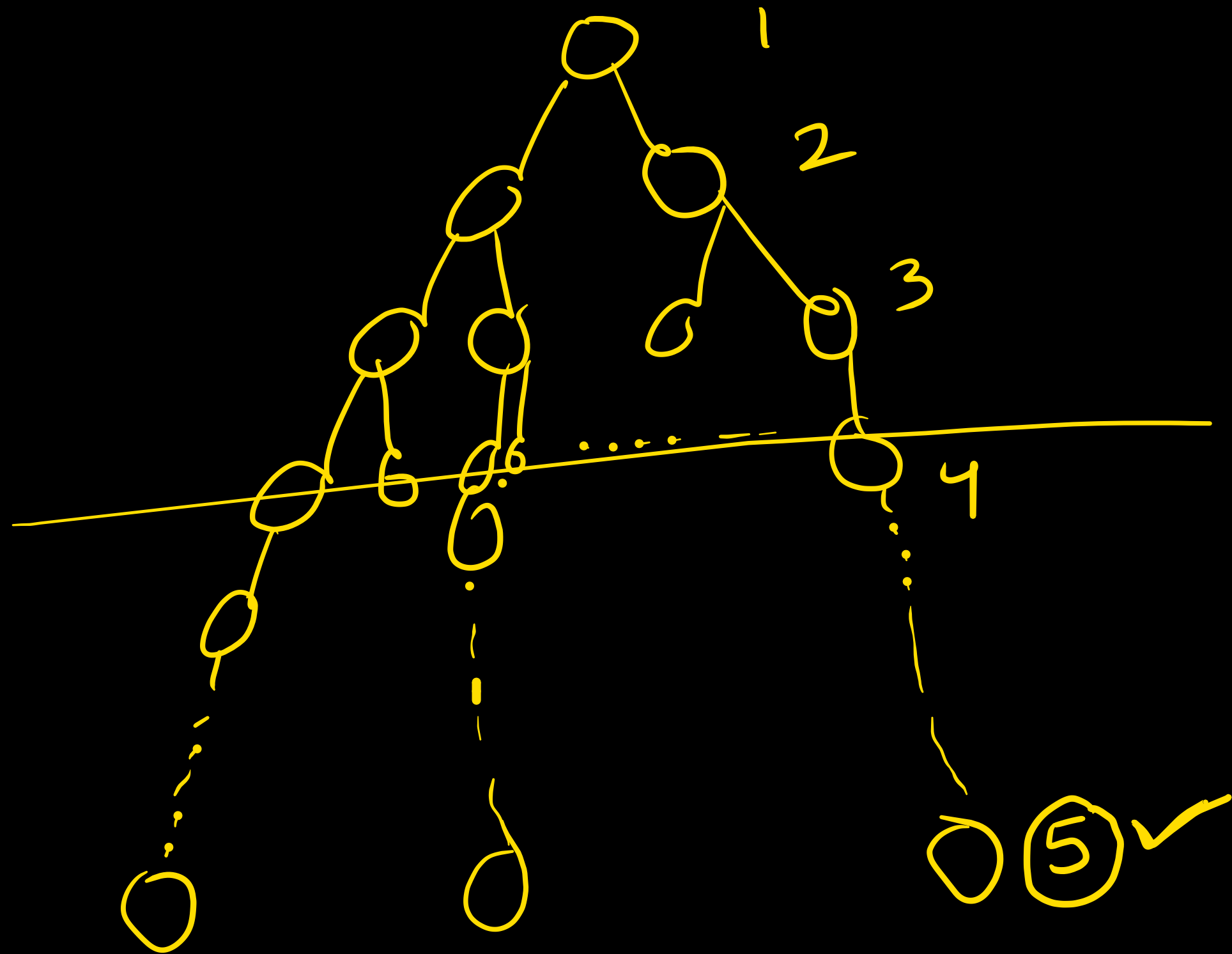
$$1 - 4$$

$$\left\lceil \frac{n}{2^{h+1}} \right\rceil = \textcircled{8} \checkmark$$

$$\left\lceil \frac{15}{2^{4+1}} \right\rceil = \textcircled{4}$$

number of nodes which are present
at level h in a complete BT

$$= \frac{n}{2^{h+1}}$$

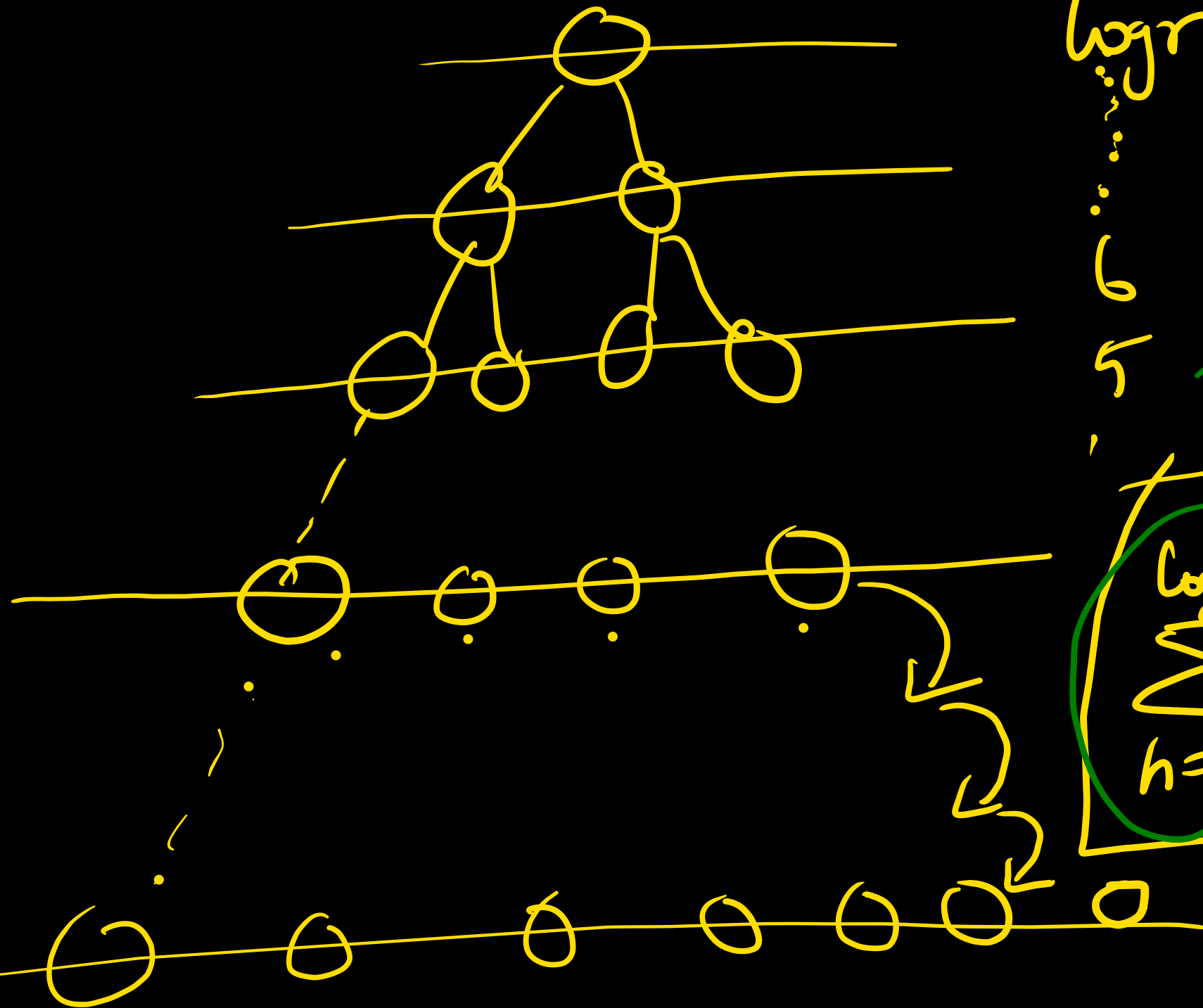


height $h = \left(\frac{n}{2^{h+1}} \right) O(h)$

total number of levels

$$= \log n$$

height
→ 0 to $\log n$



$\log n$.

no of levels
nodes per level
max heapify.

nodes

$$\sum_{h=0}^{\log n} \frac{n}{2^{h+1}} = O(h)$$

Total level
in CT

$$\sum_{h=0}^{\log n} \left\lceil \frac{n}{2^{h+1}} \right\rceil = O(h) \checkmark$$

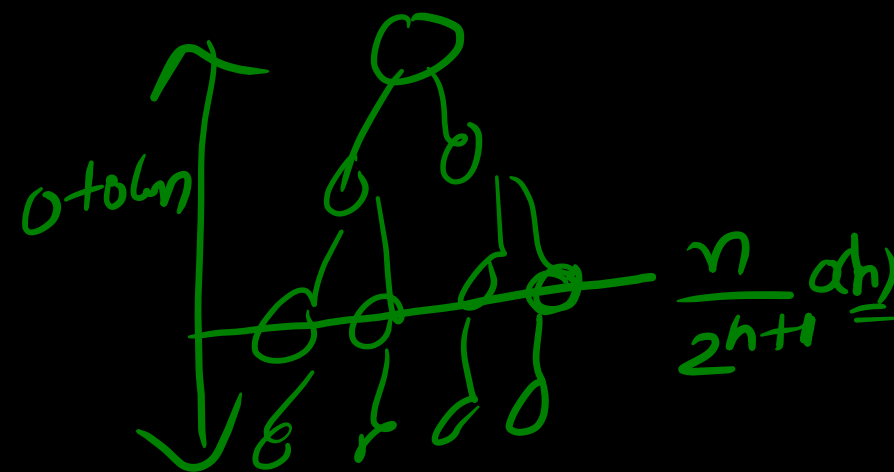
= before
0 included

$$= \sum_{h=0}^{\log n} \left\lceil \frac{n}{2^{h+1}} \right\rceil (Ch)$$

$$= \frac{Cn}{2} \sum_{h=0}^{\log n} \left(\frac{h}{2^h} \right)$$

$$\leq \frac{Cn}{2} \sum_{h=0}^{\infty} \frac{h}{2^h} = O(n) \checkmark$$

Proof is
in CLRS A.8



max height $O(\log n)$

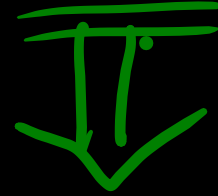
$$h = O(\log n)$$

TC $\rightarrow$ Build max heap = $O(n)$

SC $\rightarrow$ Build max heap =



Calls max heapify, one after other



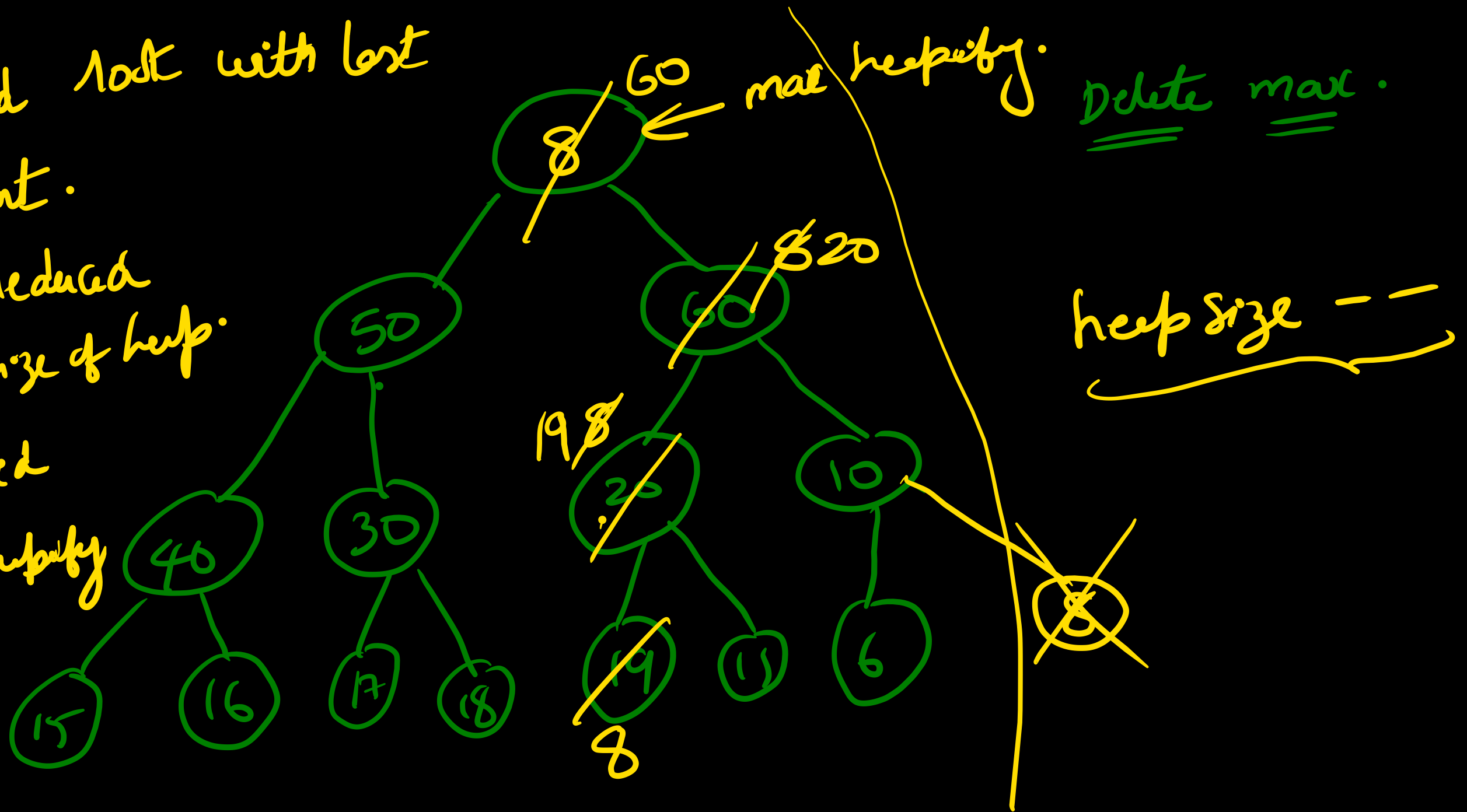
$O(\log n)$ SC.

$\therefore$ at any time only one stack is required with max size $O(\log n)$

→ Replaced node with last element.

→ reduced size of heap.

→ applied
max hubcity

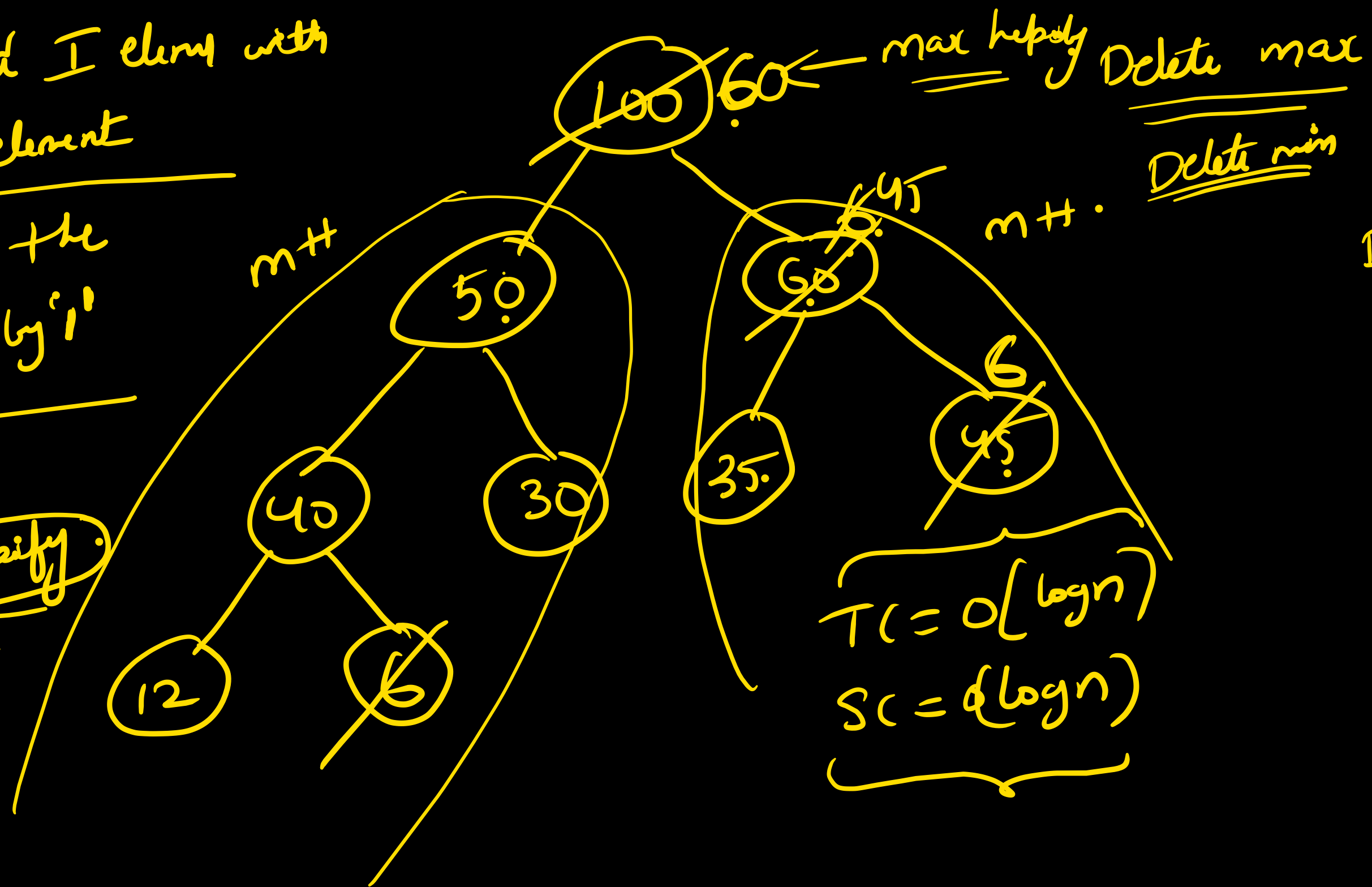


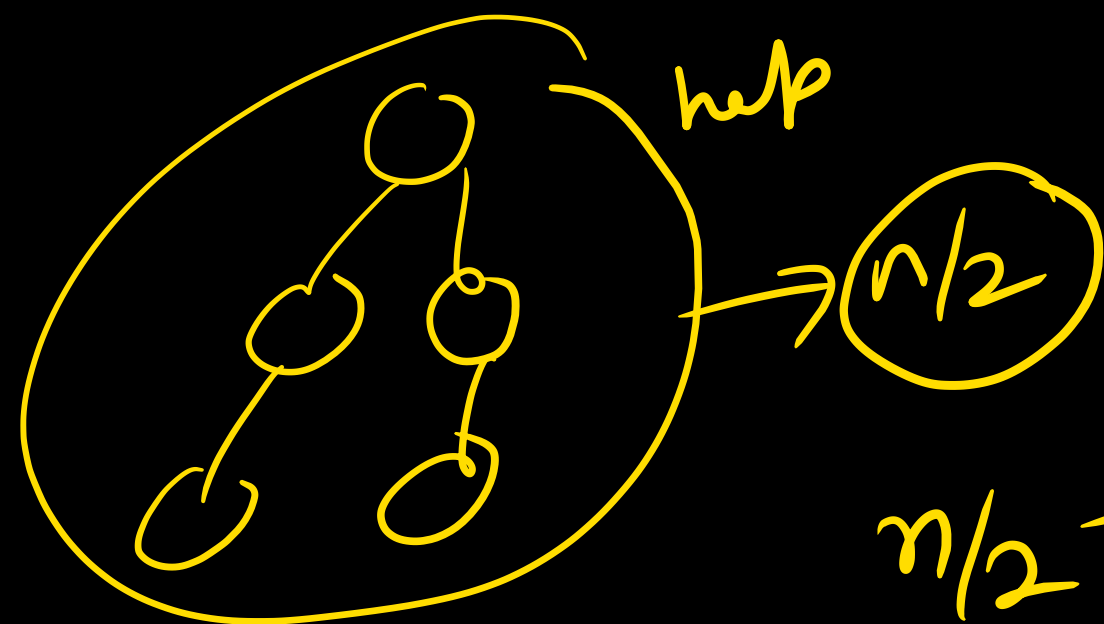
Replaced 1 elem with
last element

reduce the
heap size by '1'

apply

max heapify

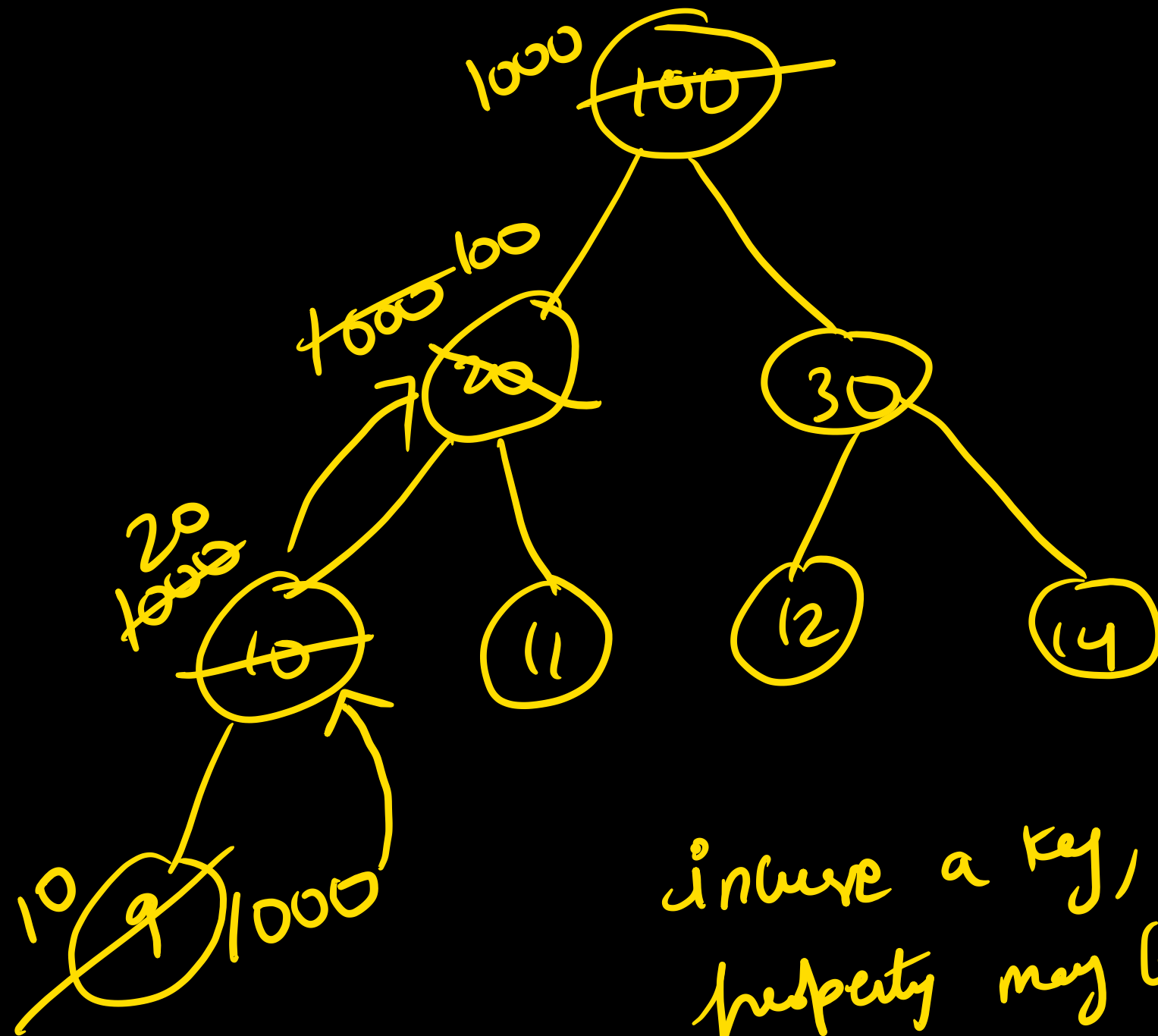




$$n/2 \rightarrow \log$$

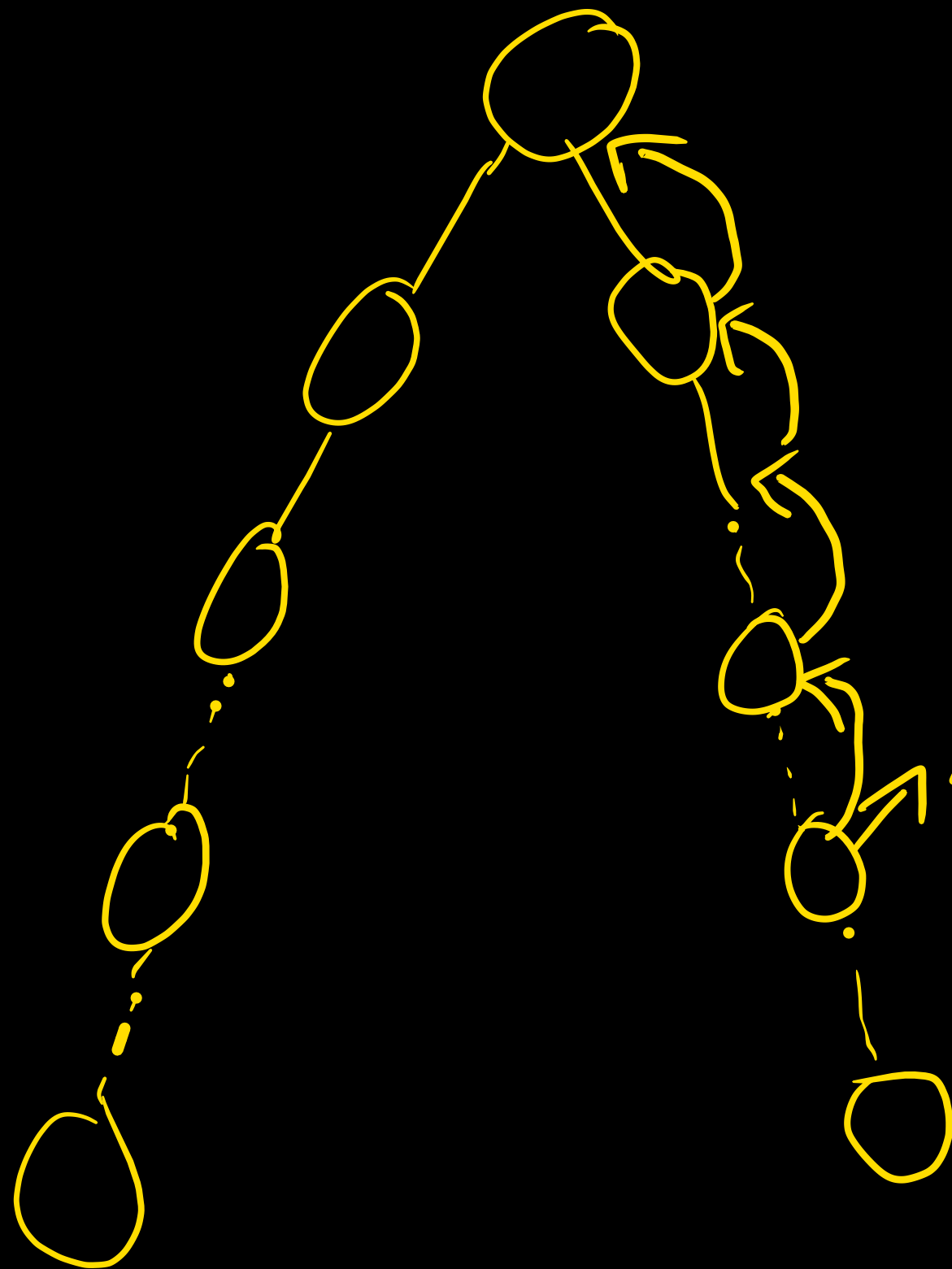
$$= \log \times \left(\frac{n}{2}\right)$$

$$\underline{\underline{O(n \log n)}}.$$



increase (Key) → element.
 9 → 1000

increase a key, max heap
 property may be violated



$O(\log n)$ ✓

cycle

Heap_incr_key(A, i, key)

{ if ($\text{key} < A[i]$)

end.

$A[i] = \text{key};$

while ($i > 1$ and $A[i/2] < A[i]$)

{ exchange ($A[i]$ and $A[i/2]$)

$i = i/2$

}

TC = $O(\log n)$
SC = $O(1)$. ✓

Heap-extract-max(A)

{ if (A.heapsize < 1)
 pf "err"

 max = A[1];

 A[1] = A[A.heapsize]

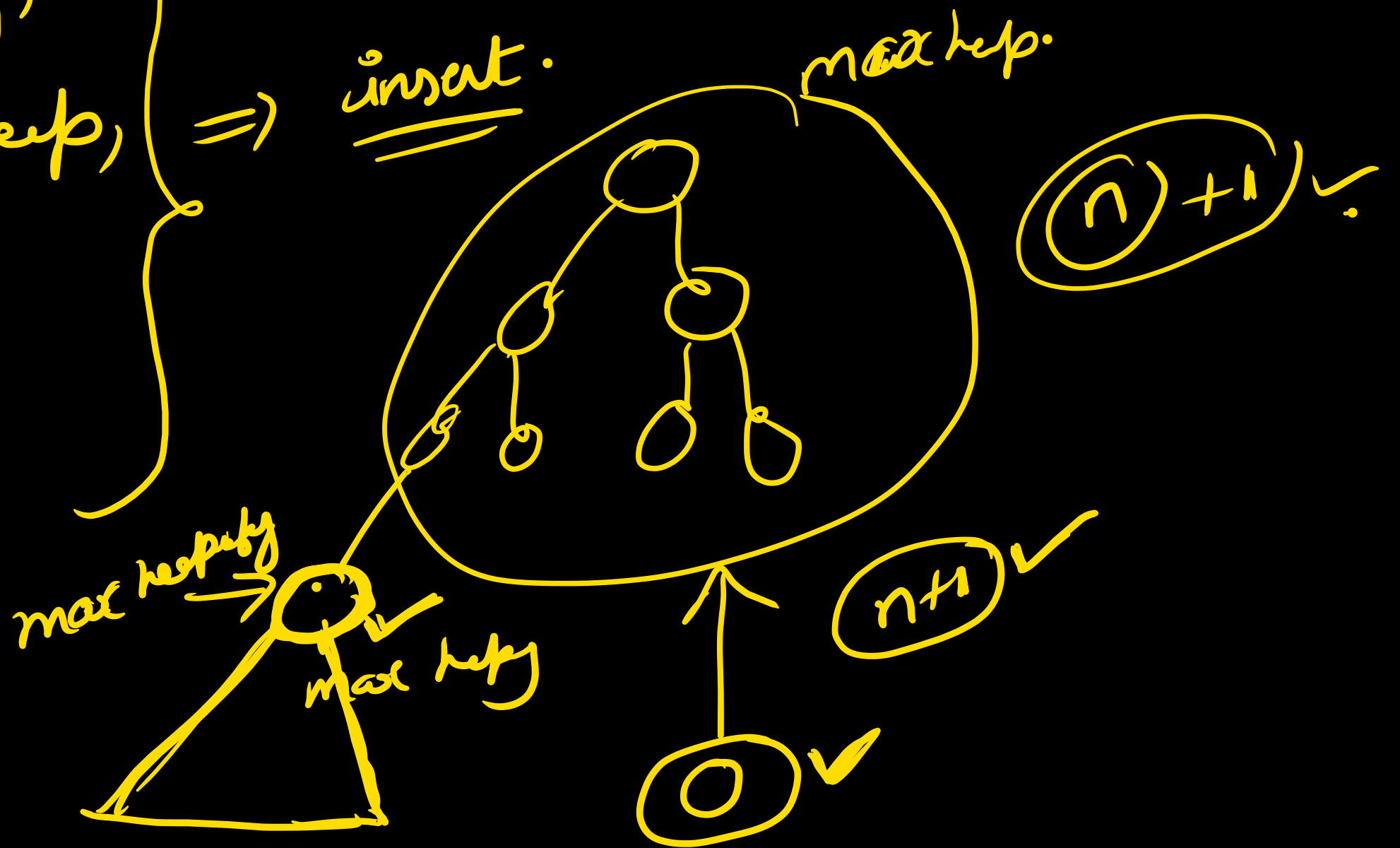
 A.heapsize = A.heapsize - 1

 max-heapify(A, 1)

 return max

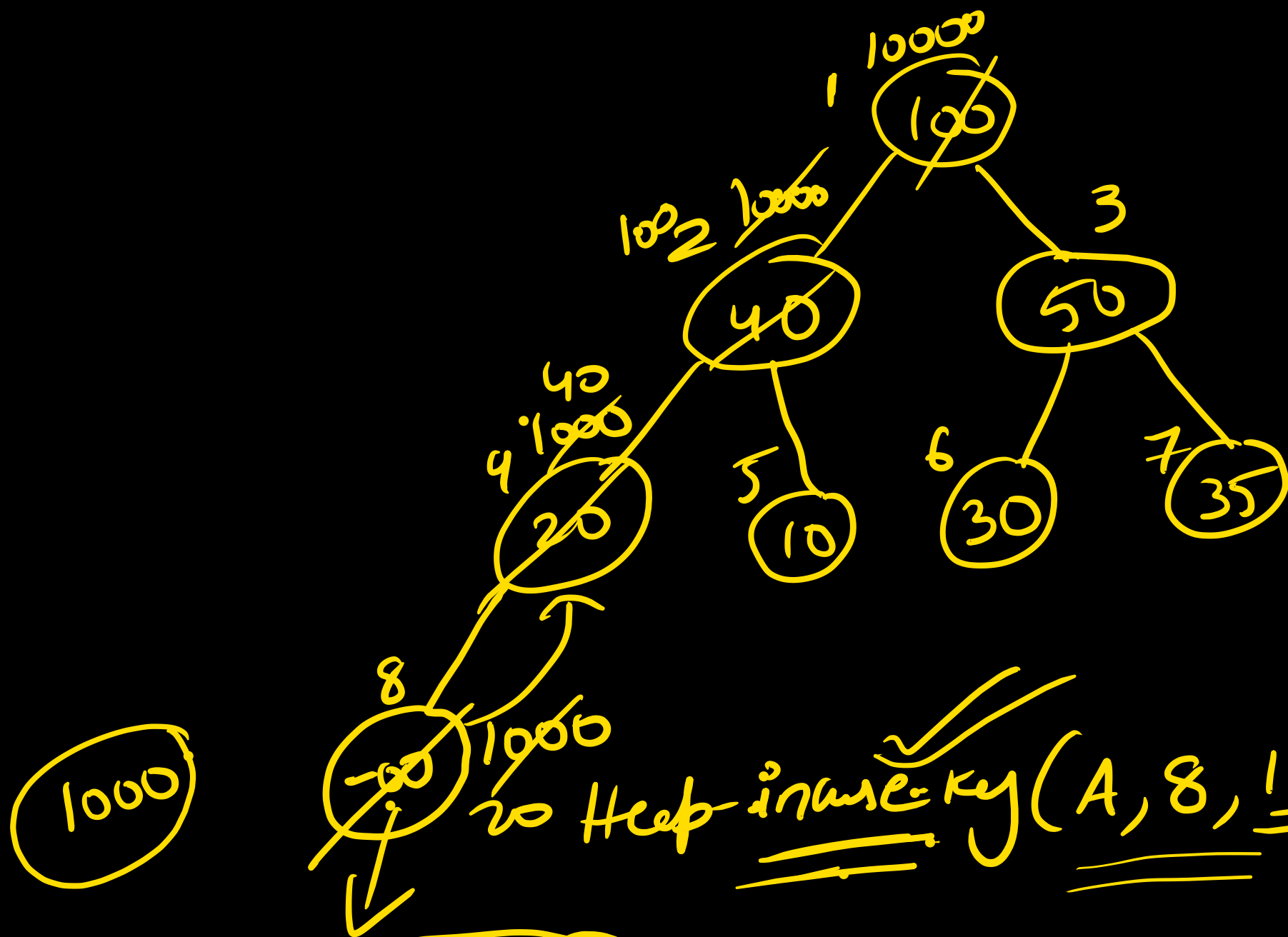
}

max heapify,
Build max heap, } $\Rightarrow$ insert.
Delete max,
insert key





$$\text{Build heap}(\underline{n+1}) = \underline{\underline{O(\log n)}}$$



$-\infty \rightarrow$ Smallest no possible.

$$T(C) = \log n.$$

$$SC = \underline{\underline{O(1)}}$$

Smallest
rep number

insert (A, x)

{

$A[\text{heapSize}+1] = -\infty;$

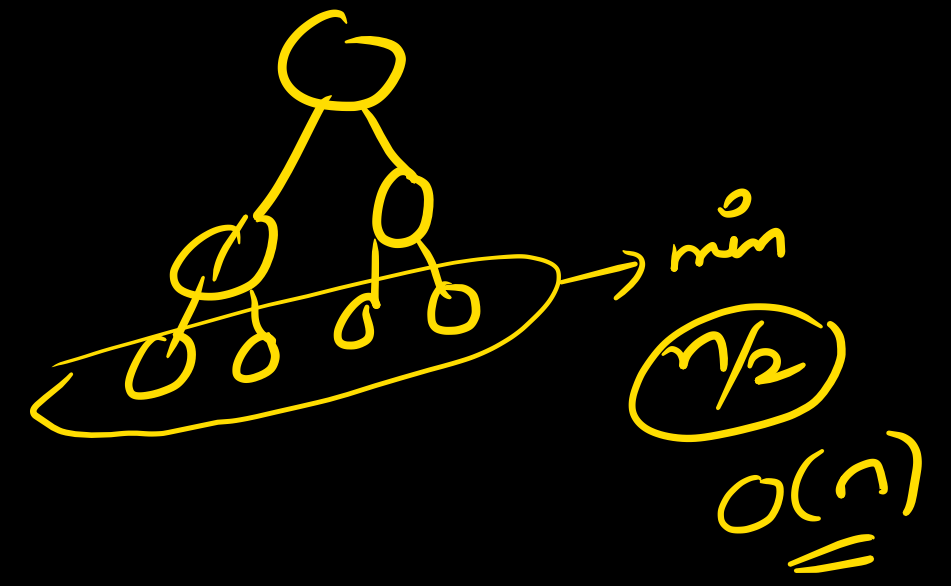
 Heap-insert-key (A, heapSize+1, x)

 heapSize = heapSize + 1;

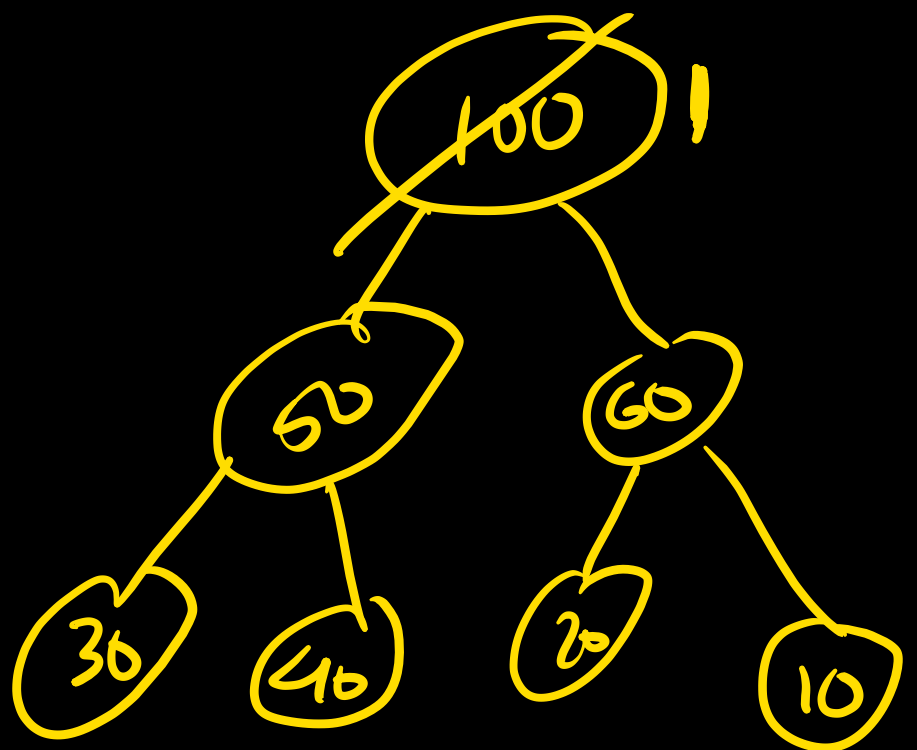
}

14

Cell heapify.



max heap:
~~max heap~~



	delete max	insert	insert key	delete key
Find max	$O(\log n)$	$O(\log n)$	$O(\log)$	$O(\log n)$
$O(1)$				
Find min				
$O(n)$ ✓				

Search for a random number
 $O(n)$

delete a random

Delete a random number

① Find the random number $\rightarrow O(n)$

② Replace it with last node and call max heapify $O(\log n)$

$O(n)$